

Experiment 5: Basic Image Handling and Processing Operations

Aim

To read an image using Python and erode the image using the Erode function.

Algorithm

1. Import the OpenCV (cv2) and NumPy libraries.
2. Read the input image using cv2.imread().
3. Create a kernel using NumPy.
4. Apply erosion using cv2.erode().
5. Display the original and eroded images.
6. Wait for a key press and close all display windows.

Python Code

```
1  import sys
2  from pathlib import Path
3  import cv2
4  import numpy as np
5
6
7  def erode_image(input_path: str, show_windows: bool = False) -> None:
8      image = cv2.imread(input_path)
9
10     if image is None:
11         raise FileNotFoundError(f"Could not read image: {input_path}")
12
13     kernel = np.ones((5, 5), np.uint8)
14     eroded_image = cv2.erode(image, kernel, iterations=1)
15
16     input_file = Path(input_path)
17     output_path = input_file.with_name(f"{input_file.stem}_eroded{input_file.suffix}")
18     cv2.imwrite(str(output_path), eroded_image)
19
20     print(f"Input image read from: {input_path}")
21     print(f"Eroded image saved as: {output_path}")
22
23     if show_windows:
24         cv2.imshow("Original Image", image)
25         cv2.imshow("Eroded Image", eroded_image)
26         cv2.waitKey(0)
27         cv2.destroyAllWindows()
28
29
30 def main() -> None:
31     input_file = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
32     show_windows = "--show" in sys.argv[1:]
33     erode_image(str(Path(input_file)), show_windows=show_windows)
34
35
36 if __name__ == "__main__":
37     main()
```

INPUT



OUTPUT



Result

The input image was successfully read using Python, and the image was eroded using the Erode function successfully.